

Tandem 16

Software Build Instructions



Overview

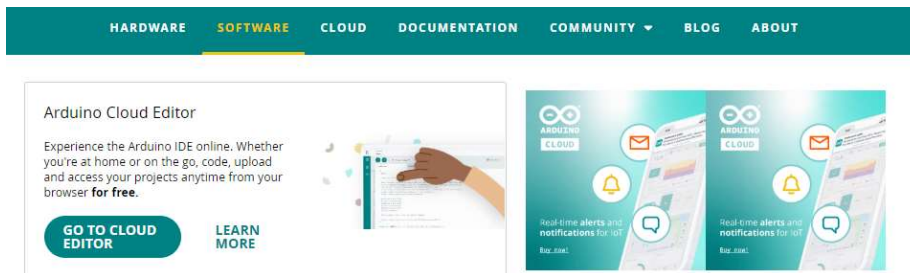
The Tandem Badge software source is provided and can be built and loaded most easily using Arduino IDE. This is the tool that was used to develop and deploy the software to the badge.

It is theoretically possible to use a loader directly to the device using the USB, but I have not done this.

These instructions walk through a complete install of Arduino IDE through loading onto the badge device. For instructions on how to assemble the device, see the assembly document.

The files required to build the badge software

- 1) Arduino IDE software – I used Windows, but this should work on Linux or Mac.



Downloads



Arduino IDE 2.3.4

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

FIGURE 1- ARDUINO DOWNLOAD

- 2) The source code. Current version is provided in git. The source code is <file-name>.ino (e.g. Badge_V1_3e.ino or similar name with .ino extension.)
- 3) Several arduino libraries are required:
 - Timers
 - WiFi
 - WiFiUdp
 - TimeLib
 - NTPClient
 - Jled
 - Time
 - AdaFruit_GFX

These libraries will be applied and created as part of these instructions.

- 4) Fonts. Fonts are provided in the github repository and will be placed as part of these instructions.
- 5) USB cable (optional 90-degree cable converter) and a PC with a USB. I used Windows, but it should work with Linux or Mac.

Step 0

Install the Arduino IDE tool

Download the Arduino IDE software. Install the software in entirety following normal software installation processes. Software is found here: [Software | Arduino](#)



FIGURE 2 - ARDUINO DOWNLOAD SELECT

Step 1

Install the board configuration

Once the IDE is installed, the boards need to be added for these specific ESP computers that are used for the badge.

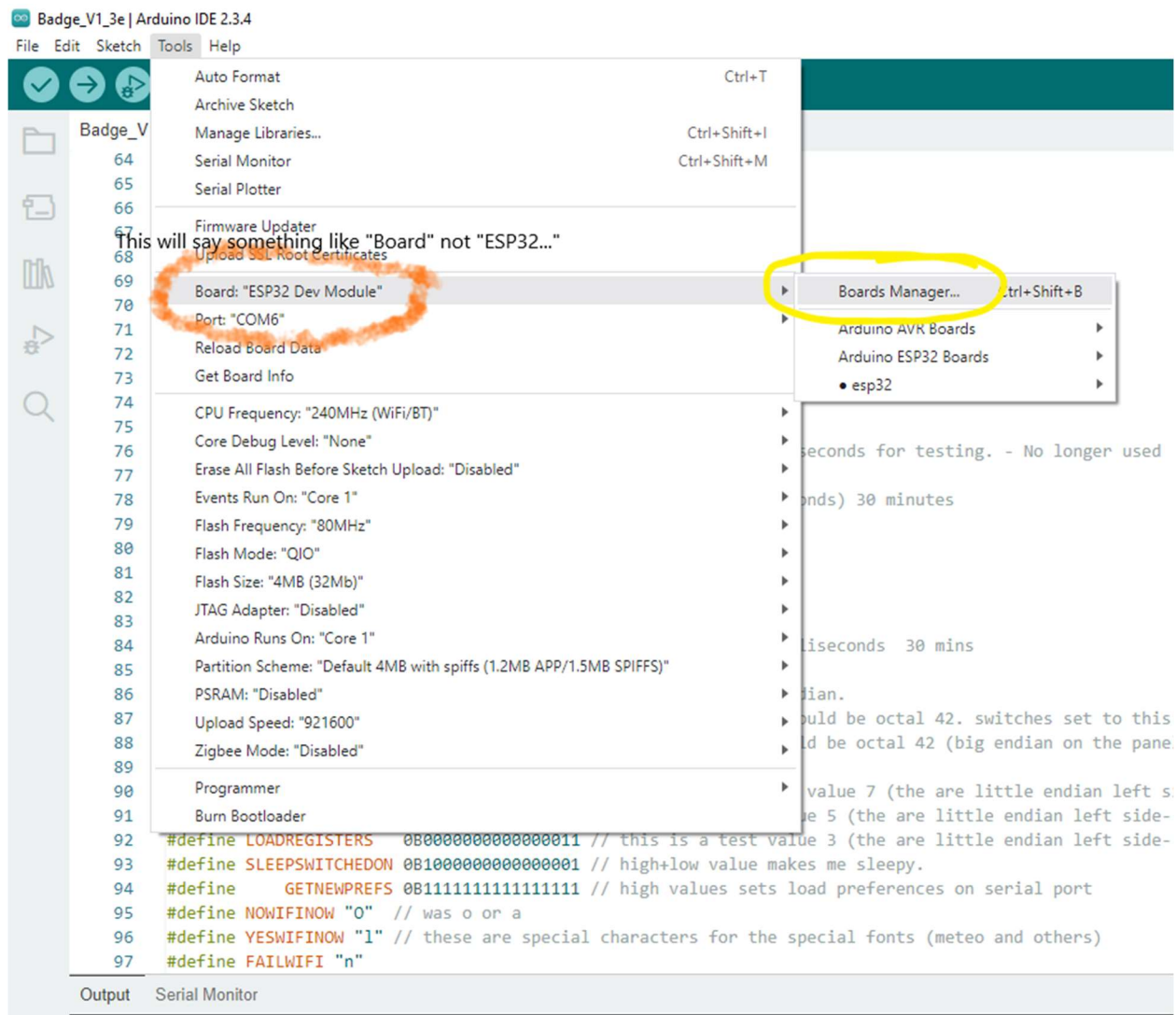


FIGURE 3 - OPEN THE BOARD MANAGER TAB

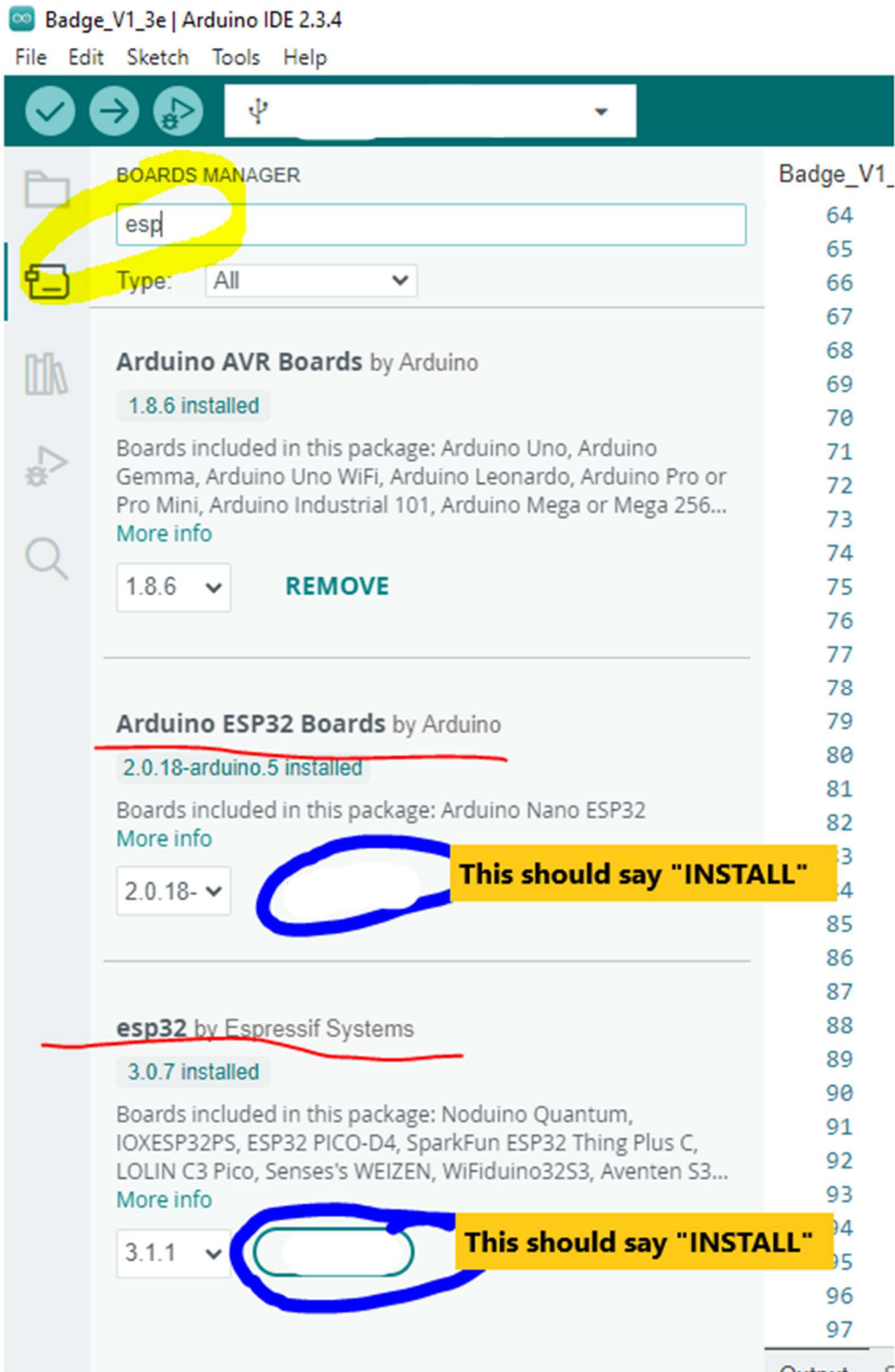
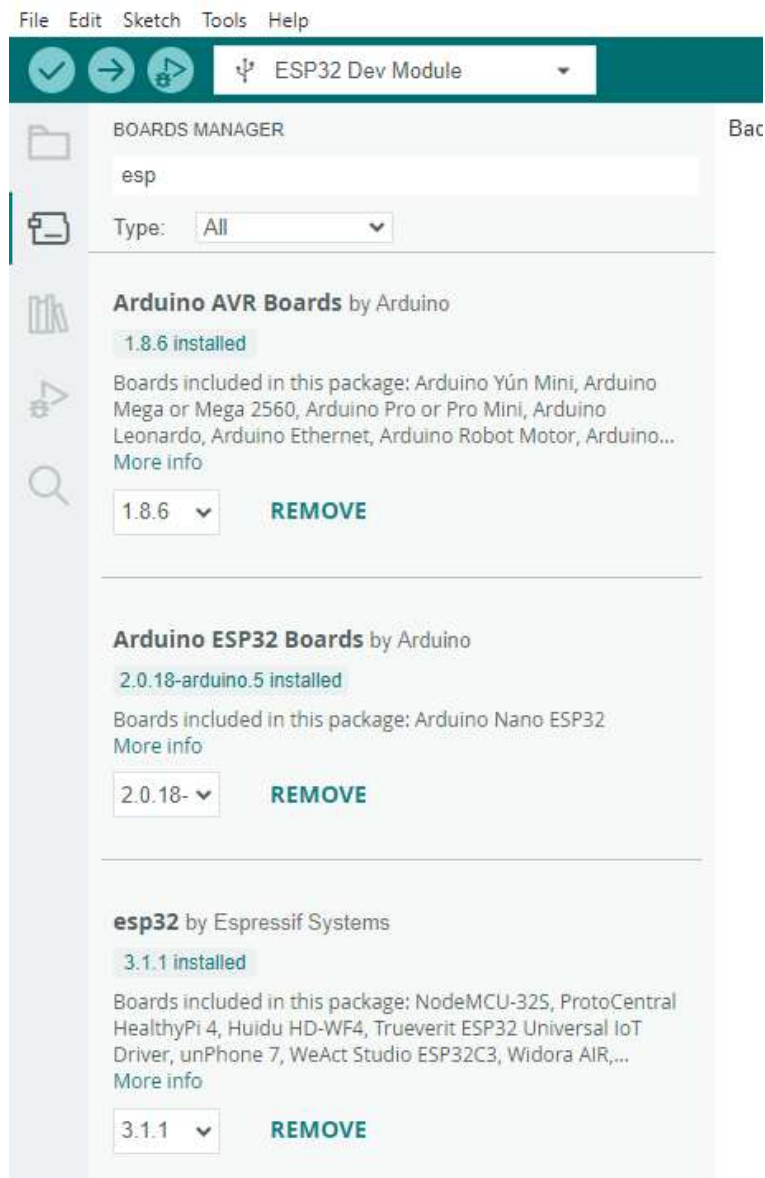


FIGURE 4 - SEARCH FOR ESP BOARD

Click on the INSTALL button for both of the ESP32 board sets. I do both, but we only need the esp32 one. You can just select esp32 by Espressif Systems. This takes several minutes. If it asks for any additional sets, approve them. (Not likely.)



Eventually you get this:

FIGURE 5 - BOARD LIBRARIES INSTALLED

Select the stair step menu Tools/Board/esp32/ESP32 Dev Board. The Tools/board value should now be ESP32 Dev Module and the board configuration parameters should all be populated (speed, debug, etc.)

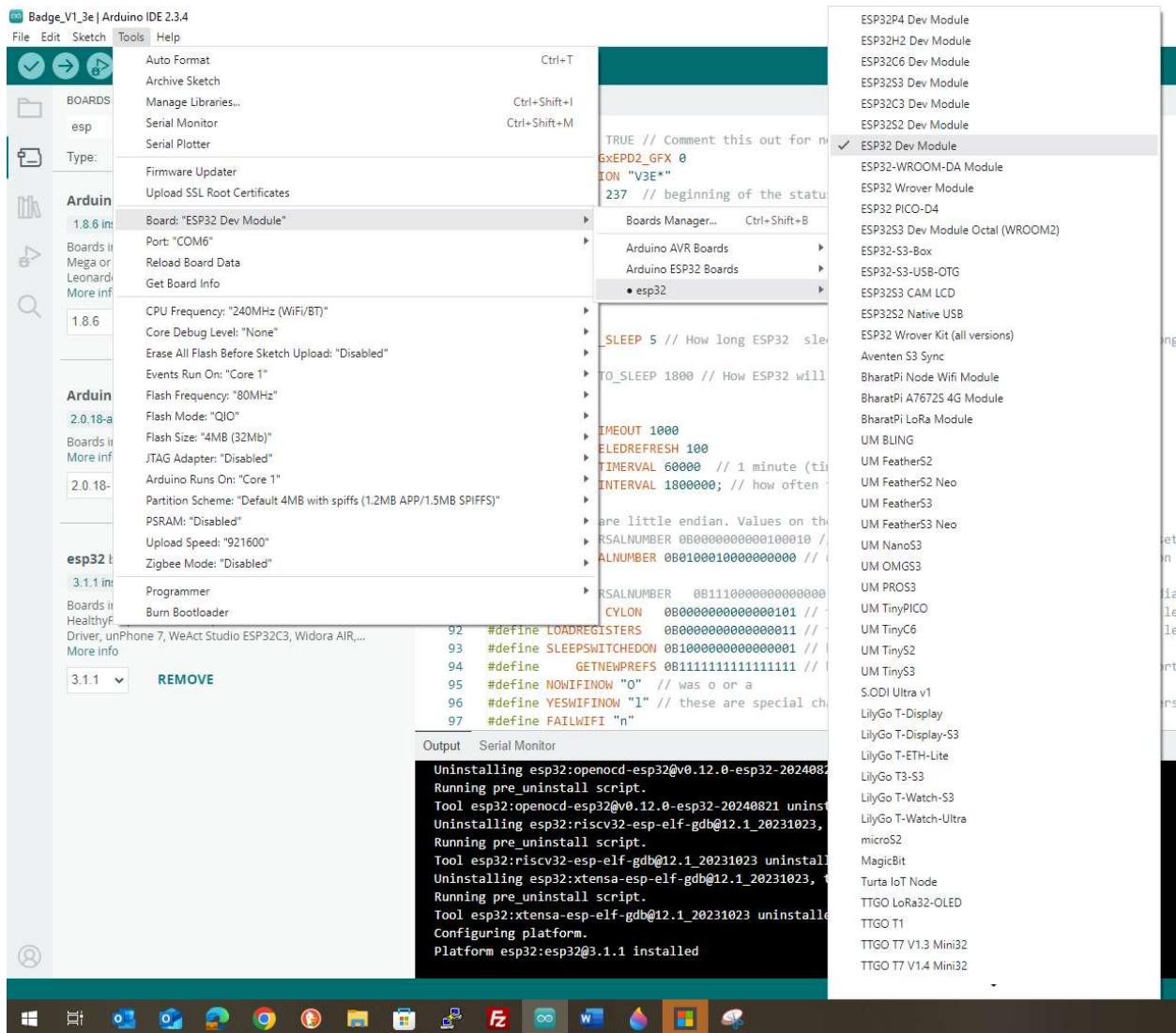


FIGURE 6 - BOARD SELECTION TEST ENABLED

Board configuration is complete. It should look similar to this when you click on the tools drop down menu without selecting anything. Maybe not exactly as shown, but should have the ESP32 Dev Module showing. That is the important part.

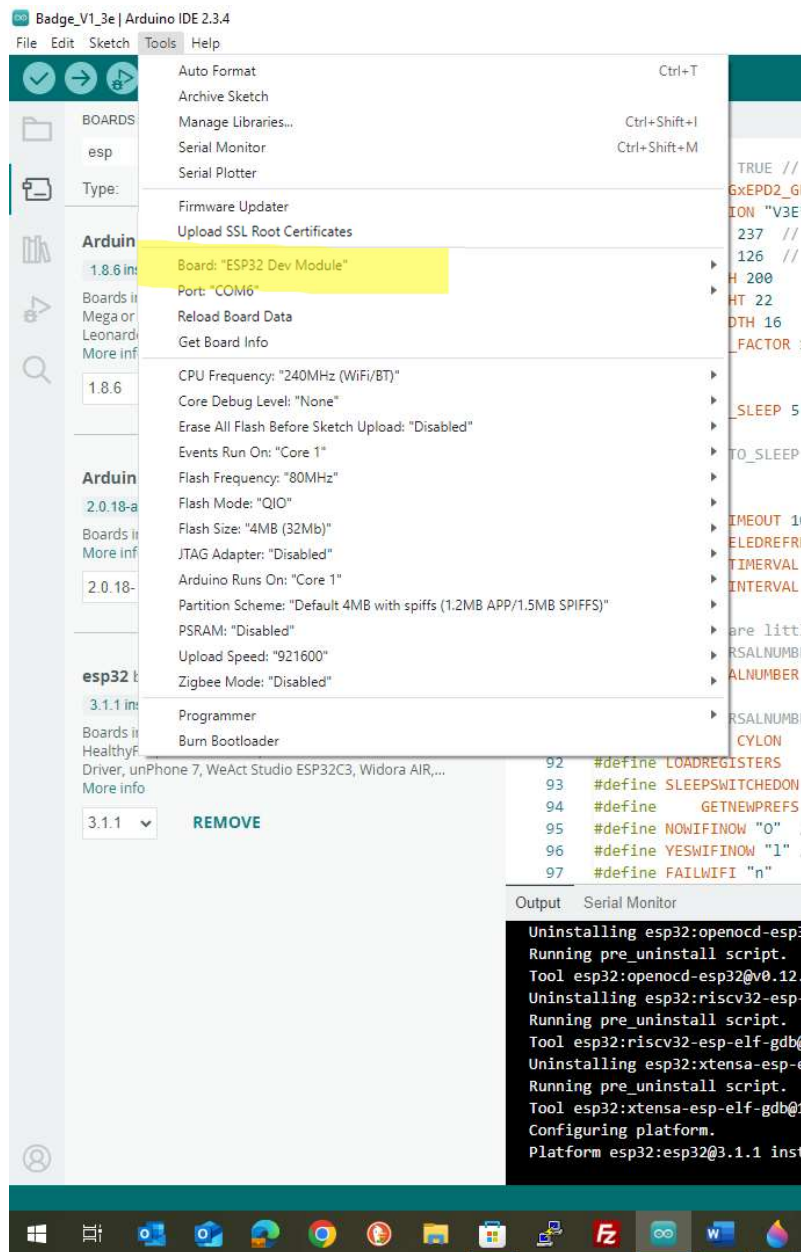


FIGURE 7 - CHECKING TO SEE IF THE BOARD IS INSTALLED.

Step 2

Install the libraries

There are two ways to install the libraries. The fastest way is to let the Arduino IDE install the library. The other way is to place the library files directly into the library folder. The libraries are in zip files.

called libraries.zip on [the github site](#). Unzip and place the folders/files directly into the Documents/Arduino/libraries directory. Once each library is placed, start/restart the Arduino IDE and it should pick up the libraries when it starts. Check this by following the second options and seeing if they reside in the tool.

The second, and best, option is to pull them using the Arduino tool.

Select stair-step menu item:

Sketch/Include libraries/manage libraries... (or CTL-SHIFT-I). This should open a side bar that has a list of libraries.

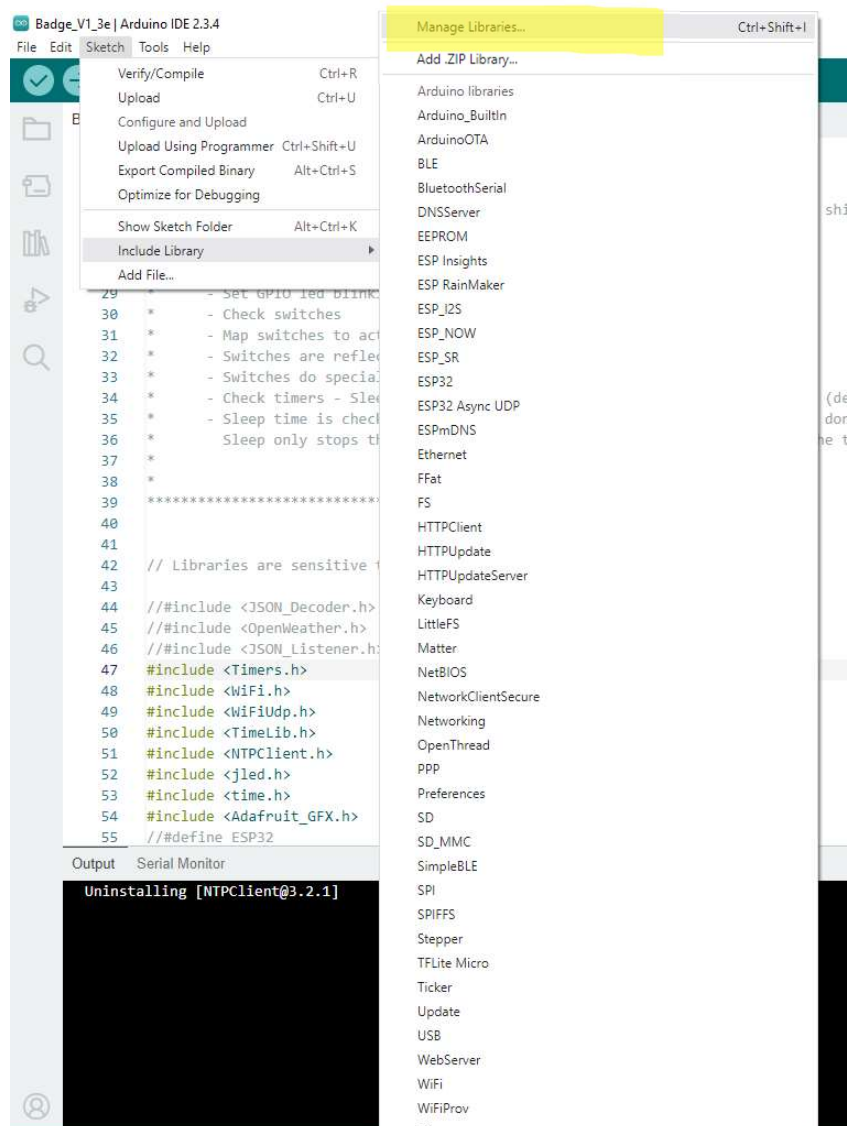


FIGURE 8 - ENABLING THE LIBRARY INSTALL MENU BAR

Install 5 Library Sets

1. NTPCLIENT
2. TIMELIB
3. ADAFRUIT GFX
4. JLED
5. TIMERS

NTPCLIENT

The libraries side bar is used to pull and install external libraries. Start by typing “ntplib” in the search space. You should see several choices. Select the one called NTPClient and select INSTALL button. The version shown might be different than the one in the image below. Let the installation finish.

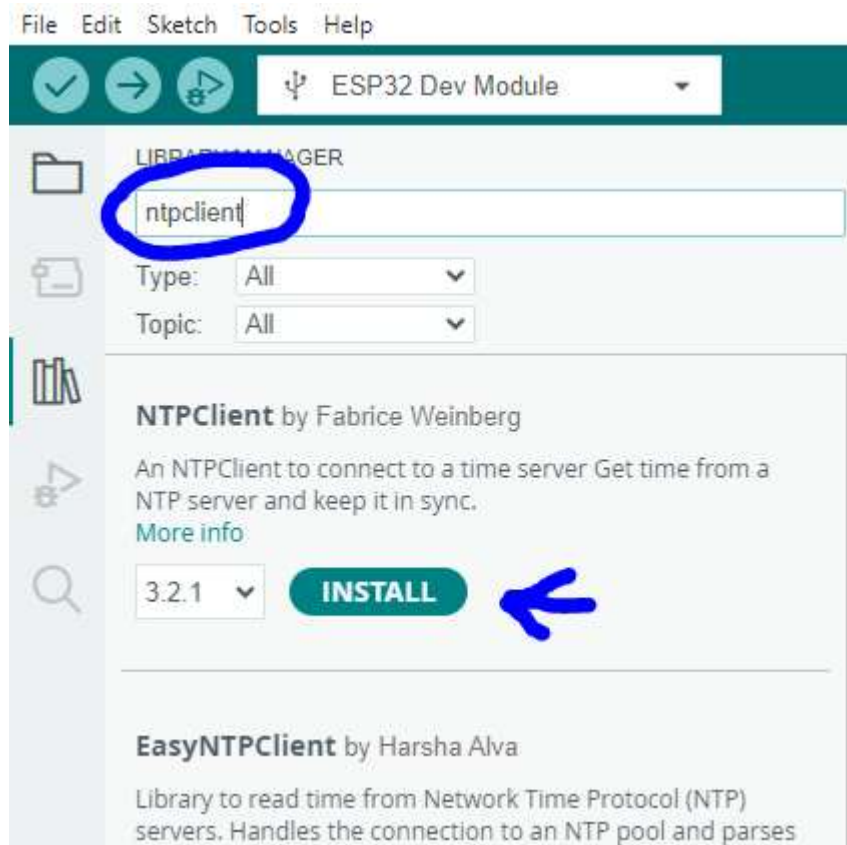


FIGURE 9 -SEARCHING FOR THE FIRST LIBRARY

The install will display in the black status window below.

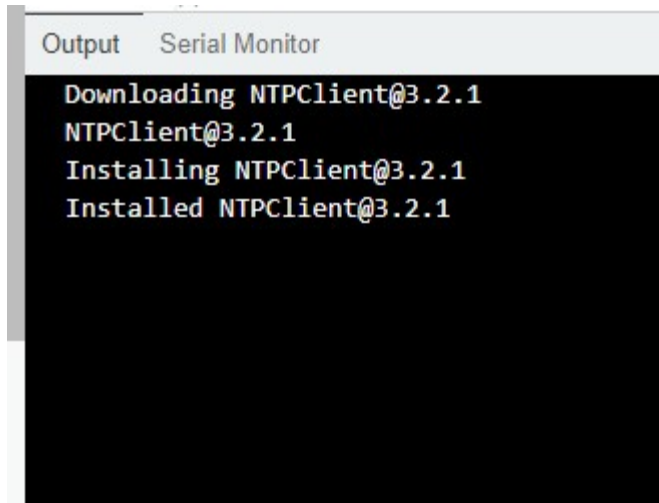


FIGURE 10 - LOG OF THE INSTALLATION (FOR MONITORING ONLY)

REPEAT this step for each of the libraries called:

TIMELIB

Search “Timelib” (by Michael Margolis)

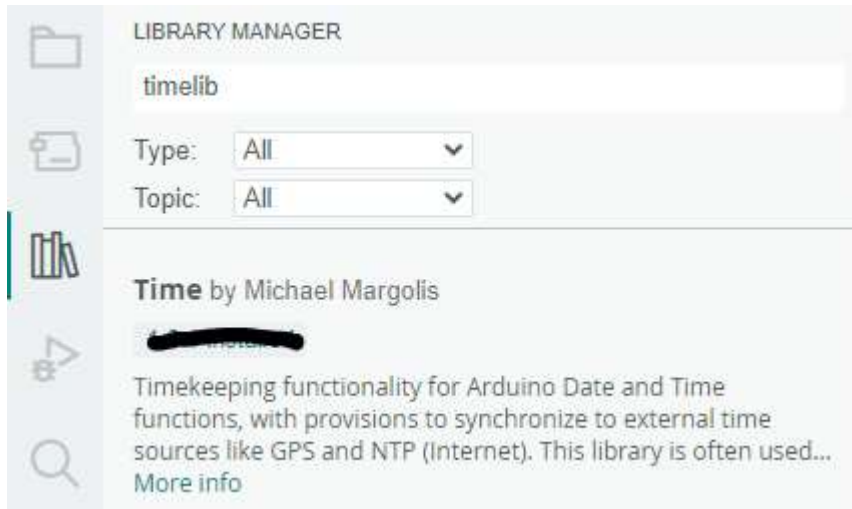


FIGURE 11- TIMELIB - MAKE SURE YOU GET THIS ONE!

ADAFRUIT GFX

Seach “Adafruit GFX Library”

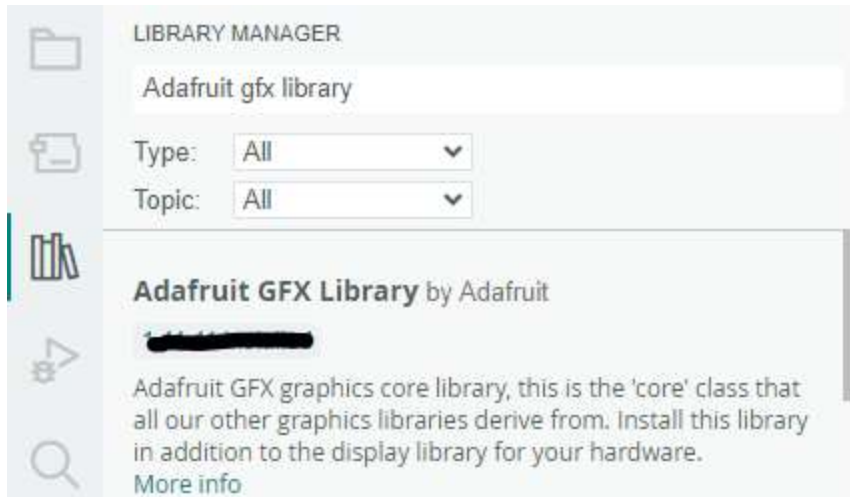


FIGURE 12 - ADAFRUIT GFX

JLED

Search “JLED” by Jan Delago

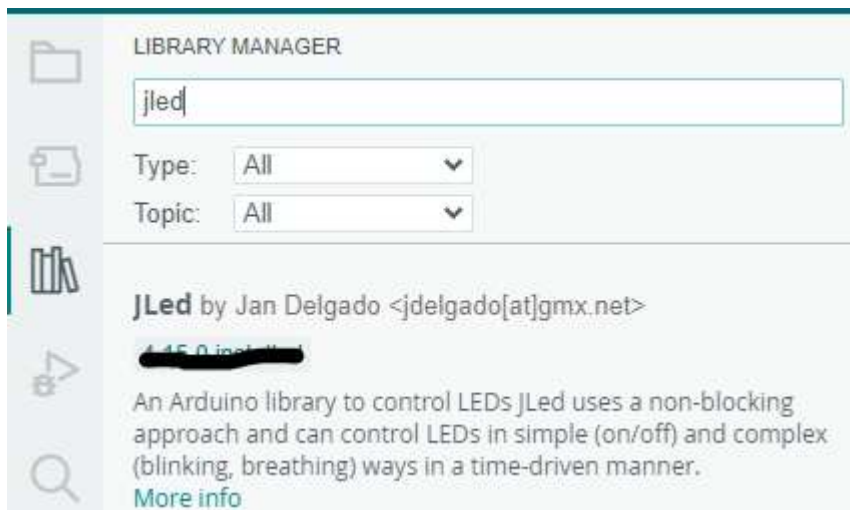


FIGURE 13 - JLED LIBRARY

TIMERS

Search “Timers” by Centaq – Be careful to get this exact one. There are several to choose from. Get this one.



FIGURE 14 - TIMERS LIBRARY BY CENTAQ

The other libraries should be already available via the ESP board platform libraries. No need to look for those.

Step 3

Configure and connect the Display computer via USB

It is a good idea to stop and restart the Arduino IDE after updating everything above.

Please the USB cord into the display device. Use the 90 degree converter on the display side if desired/necessary. Plug the USB into the computer. You may see a device download. Probably not.

Once connected, You need to make sure of two things:

- 1) Make sure that the proper BOARD is selected in the IDE. Stairstep menu selection:
Tools/Board/esp/ESP32 Dev Module

It is critical which board type is used. Use ESP32 Dev Module as the board type. If you select the wrong board type, things will appear to work during the compile, but the devices will not function properly, or more likely, strange errors will happen during the compile-and-load step below.

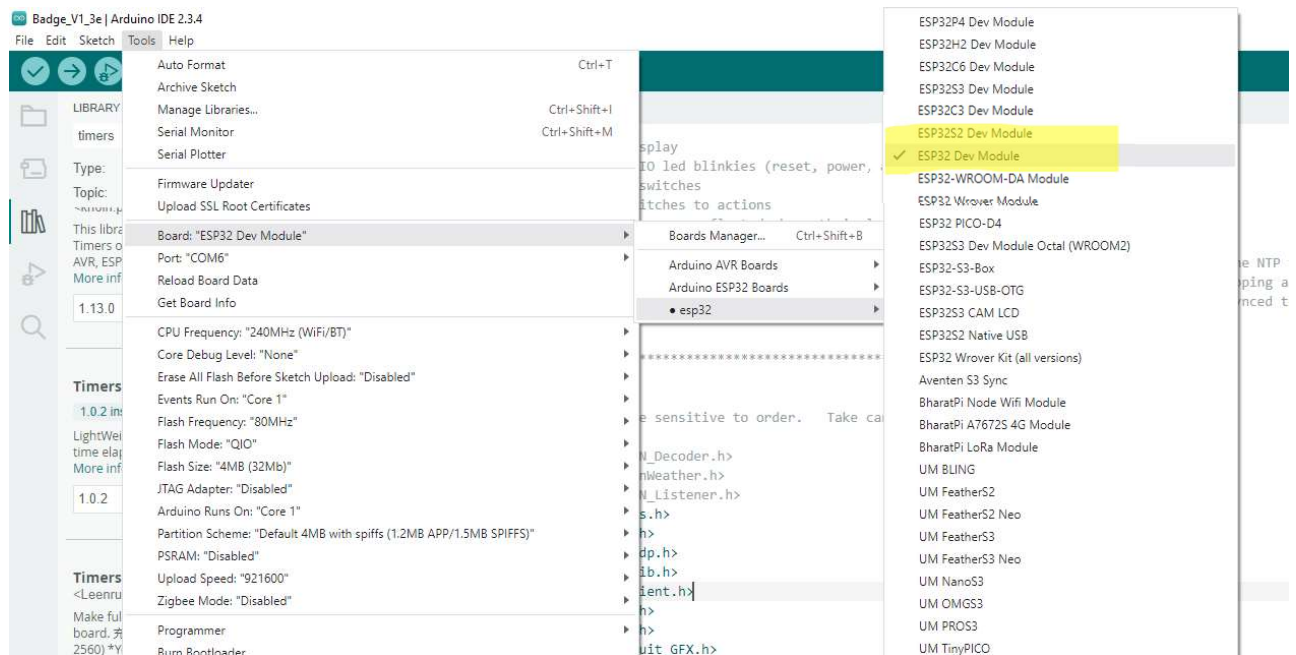


FIGURE 15 - SELECTING THE DEVICE

2) Select the appropriate Serial communications port and configuration.

Turn the device on now if it is not already on. It only needs to be on for the software to load, but it is a good time to turn it on so that you do not forget later.

Power switch is located at the lower bottom of the underside of the display as part of the electronics of the display board. You will also see the reset buttons. We do not use the rest buttons for any of this.

POWER ON slide switch to the LEFT. POWER OFF slide switch to the right.

In the Arduino IDE software, select Stair-step menu **TOOLS/Port/<the COM port >**

In the example below, <the COM port> is COM3. Select that – whatever value is presented to you as available for the device. (Sometimes it even shows as Lilygo display.) You can double check this configuration doing just a **TOOLS** drop-down and looking at the value for Board and for Port. They should both be now set to appropriate values.

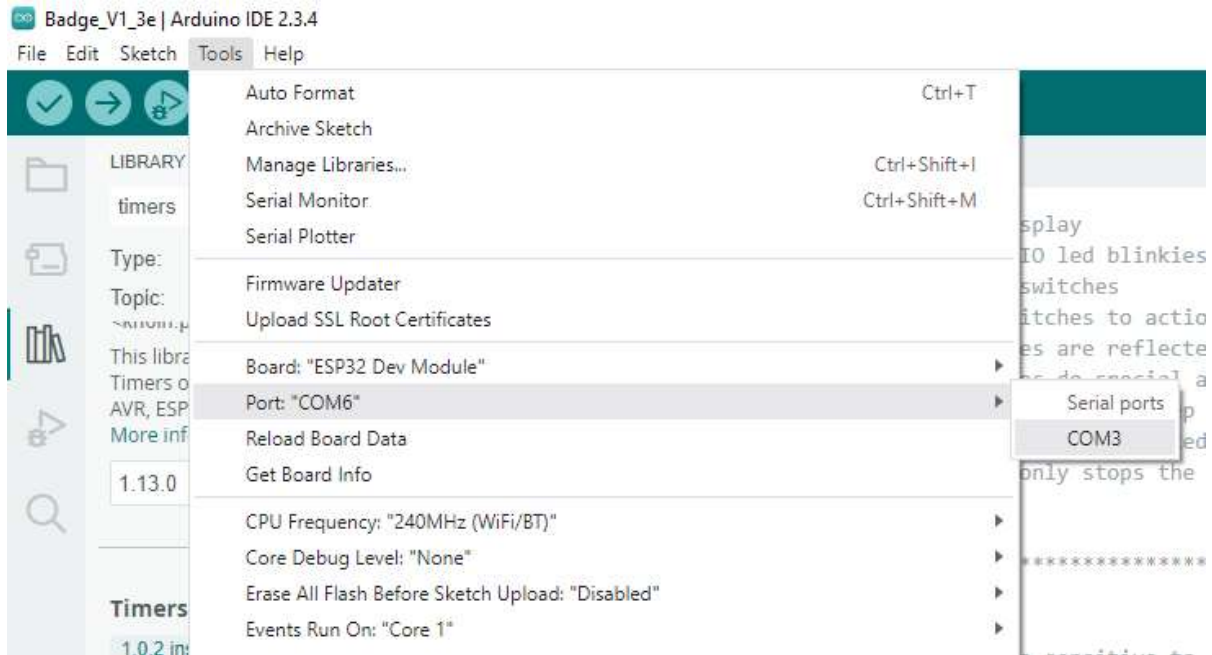


FIGURE 16 - CHOOSE THE ACTIVE COM PORT

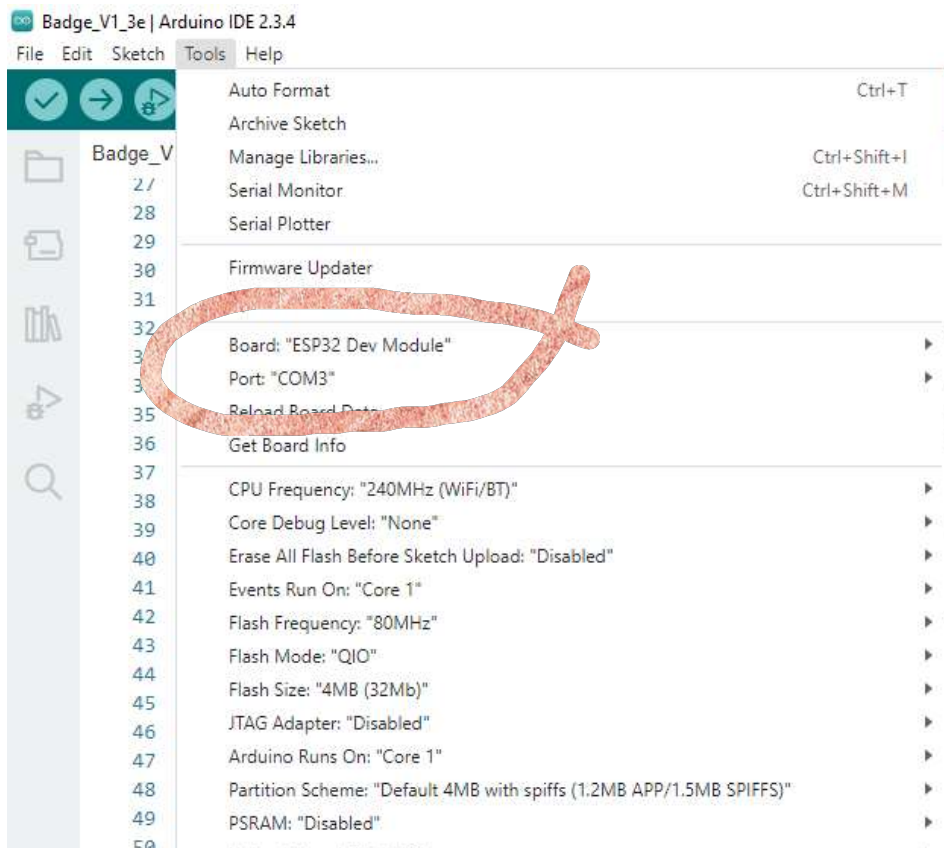


FIGURE 17 - CHECKING THE COM PORT AND BOARD TYPE (COM VALUE VARIES).

Step 4

Compile and load the software

This is the most exciting step. It all comes together now. If anything does not work in this step, go back and re-do/recheck all of the above software installs. Everything you just did is “re-doable” as many times as necessary to get it right.

Best practice is to place the entire folder called Badge_V1_3e into Documents/Arduino (default location of all Arduino source code. aka “sketches”).

This is made easiest by just pulling the zip file with everything in it. Unzip the file called Badge_V1_3e.zip into Documents\Arduino\. Be sure to place the entire folder with all files underneath.

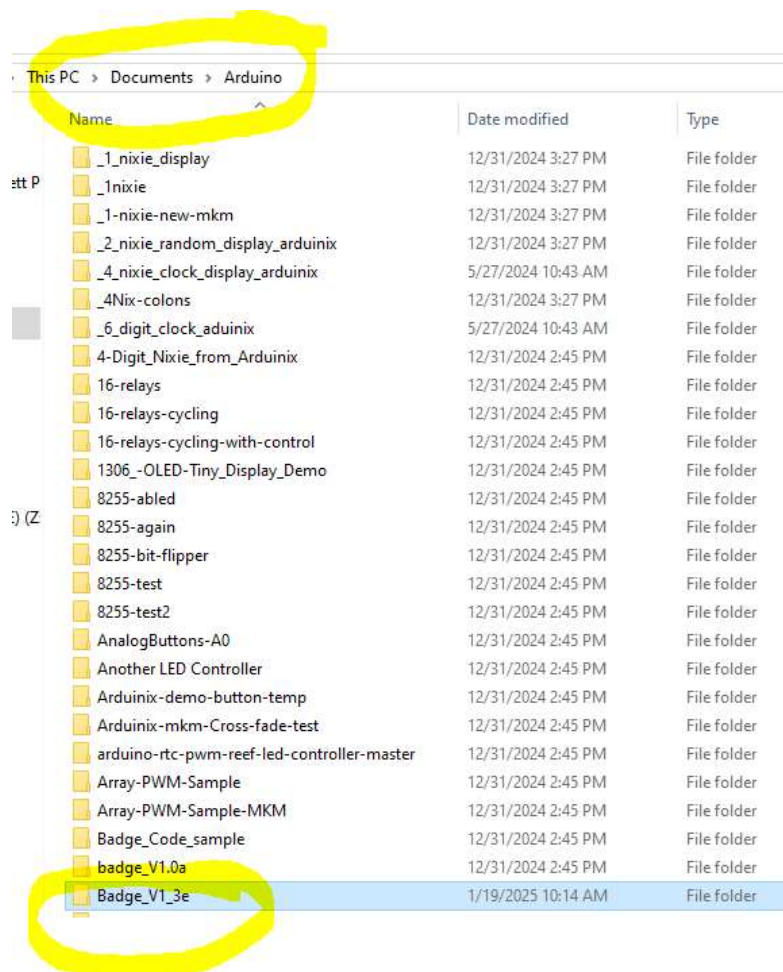
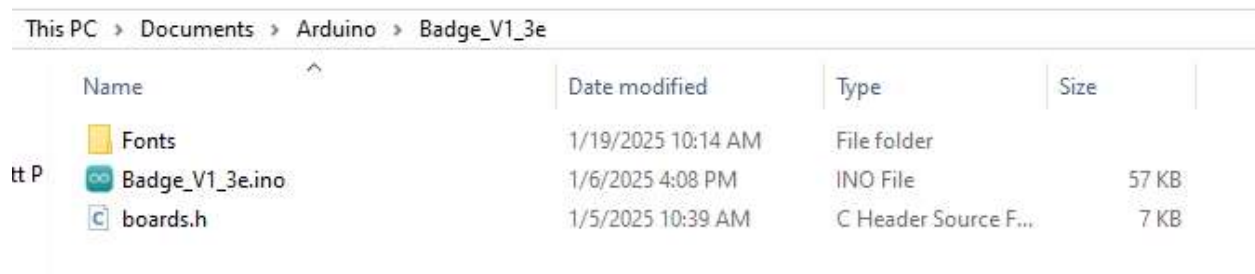


FIGURE 18 - DEFAULT LOCATION OF THE SOURCE SKETCH FILES.

It is critical that these three files be inside this folder.

- 1) Fonts folder – Several files inside. These are obviously the fonts used in the display.
- 2) Badge source sketch file which is a .ino file. It should be associated to the Arduino IDE as shown.
- 3) A file call boards.h. This file is imperative in order for the compiler to resolve references to the display type.

All three must reside inside the same directory as the .ino file. The folder itself can reside anywhere, but it is recommended to use the default of Documents\Arduino.



This PC > Documents > Arduino > Badge_V1_3e				
Name	Date modified	Type	Size	
Fonts	1/19/2025 10:14 AM	File folder		
Badge_V1_3e.ino	1/6/2025 4:08 PM	INO File	57 KB	
boards.h	1/5/2025 10:39 AM	C Header Source F...	7 KB	

FIGURE 19 - CONTENTS OF THE BADGE SKETCH FOLDER

Compile command and result

The compile/build can be done as a separate step or combined with the upload command. For simplicity we will use the upload command which does both steps.

Use File/Open... to chose the .ino sketch file mentioned above. Use the standard navigation to point to Documents\Arduino\Badge_V1_3e (or wherever you placed it above).

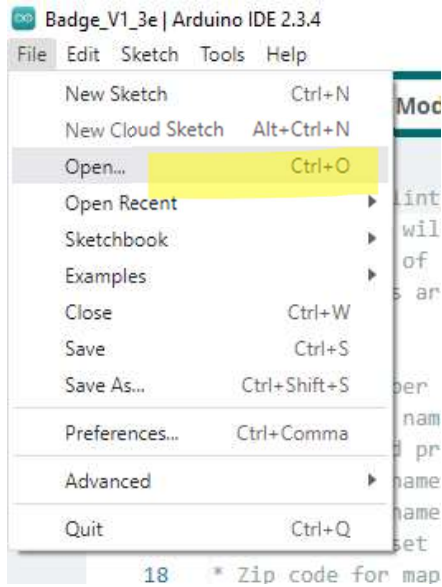


FIGURE 20 - STANDARD FILE OPEN...

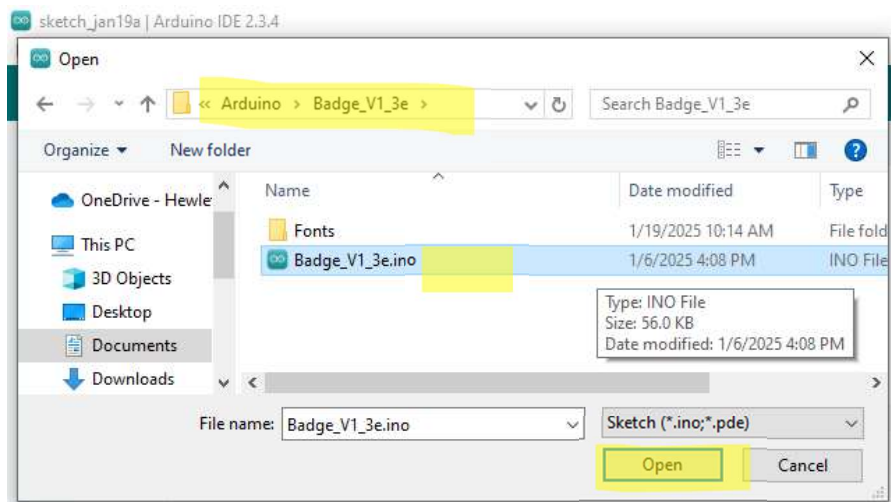


FIGURE 21 - SELECT SOURCE SKETCH FILE

Once opened you should see the source.

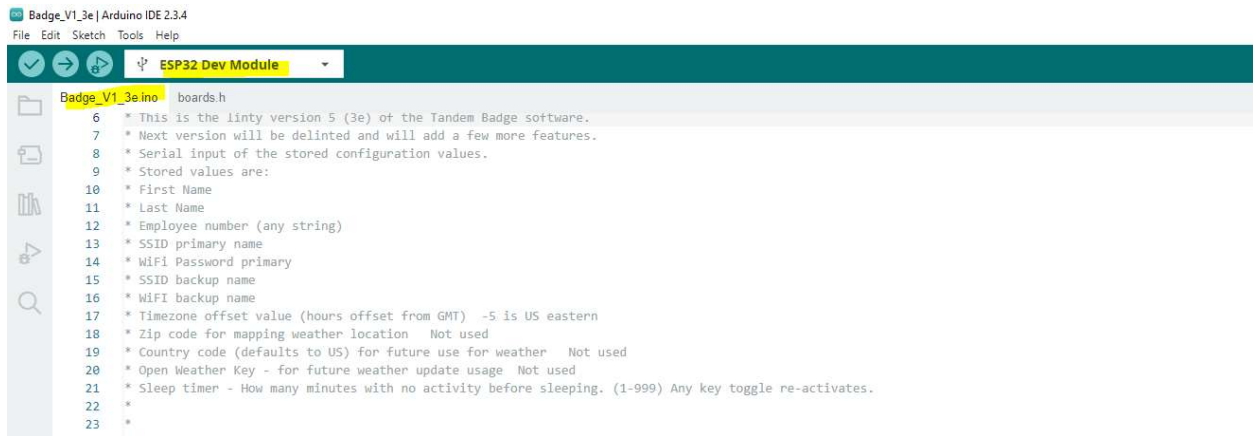
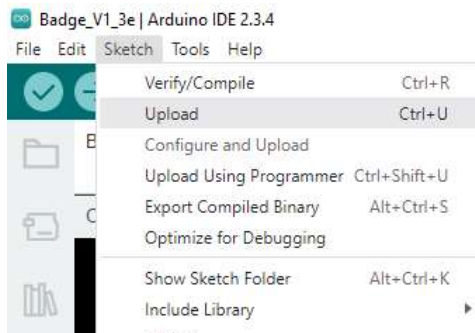


FIGURE 22 - BADGE SOURCE SHOWN - READY TO COMPILE AND LOAD

Double check that the correct device is showing (highlighted in the above diagram) and check that the COM port is connected and the device is turned on. Ignore any behavior of the device. Display behavior is somewhat random since the device has no software on it yet. It is safe to ignore the display and the led activity.

If everything looks good, and the USB cable is a data cable (not just a power cable), proceed.

Select Sketch/Upload from the menu (or press CTL-U)



The output should start. It will take several seconds to several minutes The first compile will take several MINUTES. Do not fret. The tool caches the builds. But since this is a fresh new build, it must resolve and compile a lot of stuff. Eventually, you will see some significant output.

Output

```
Sketch uses 1141540 bytes (87%) of program storage space. Maximum is 1310720 bytes.
Global variables use 52448 bytes (16%) of dynamic memory, leaving 275232 bytes for local variables. Maximum is 327680 bytes.
esptool.py v4.8.1
Serial port COM4
Connecting.....
Chip is ESP32-D0W0Q6-V3 (revision v3.1)
Features: Wifi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: a8:42:e3:3b:d6:28
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 921600
Changed.
Configuring flash size...
Flash will be erased from 0x00001000 to 0x00007fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x0000e000 to 0x0000ffff...
Flash will be erased from 0x00010000 to 0x00126fff...
Compressed 24976 bytes to 15952...
Writing at 0x00001000... (100 %)
Wrote 24976 bytes (15952 compressed) at 0x00001000 in 0.6 seconds (effective 311.6 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 146...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.1 seconds (effective 371.7 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 47...
Writing at 0x0000e000... (100 %)
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.1 seconds (effective 491.0 kbit/s)...
Hash of data verified.
Compressed 1141664 bytes to 636923...
Writing at 0x00010000... (2 %)
Writing at 0x0001c62b... (5 %)
Writing at 0x00026d40... (7 %)
Writing at 0x0003399e... (10 %)
Writing at 0x0003b901... (12 %)
Writing at 0x0006760d... (15 %)
Writing at 0x0006cd1f... (17 %)
Writing at 0x00072a6e... (20 %)
Writing at 0x00078462... (23 %)
Writing at 0x0007dd83... (25 %)
Writing at 0x000834af... (28 %)
```

FIGURE 23 - OUTPUT FROM COMPILE AND LOAD PART 1

And eventually...

```
Writing at 0x0011040e... (94 %)
Writing at 0x0011bcd5... (97 %)
Writing at 0x001215e8... (100 %)
Wrote 1141664 bytes (636923 compressed) at 0x00010000 in 10.6 seconds (effective 865.0 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

FIGURE 24 - SUCCESSFUL COMPILE AND LOAD

Once this is completed, the device will reboot and you will see empty/default display. The switches will behave as [documented](#) but the name and Wifi does not work yet. Play as much as you want with the device or can continue. If you are going to continue, leave the device connected to the IDE for the next steps.

Step 5

Load personal configuration information into the memory

With the software loaded and the device STILL connected to the Arduino IDE, open the serial terminal inside of the IDE. Select from the menu (CTL-SHIFT-M), Tools/Serial Monitor...

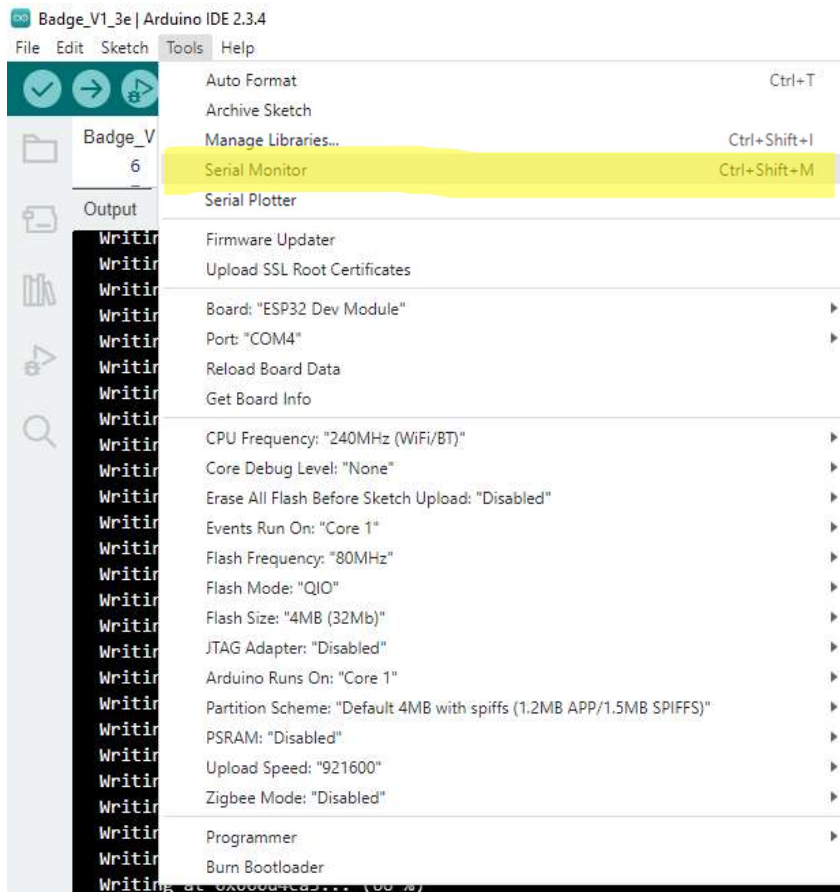


FIGURE 25 - SELECT SERIAL MONITOR FROM IDE

You should see a few odd messages from the device. Likely they will be garbled.

Important: Set the serial port communication speed to 115200.

The serial monitor should switch to white at the bottom of the screen and on the far-right side there is a drop down that sets the communication port speed. Set to 115200.

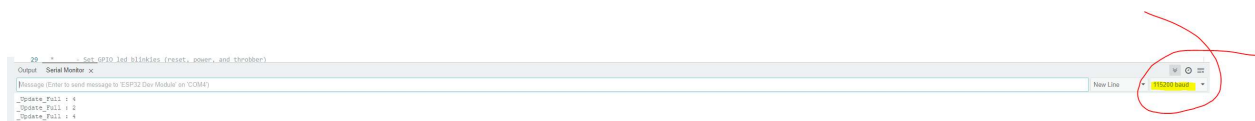




FIGURE 26 - SET THE COMMUNICATION SPEED

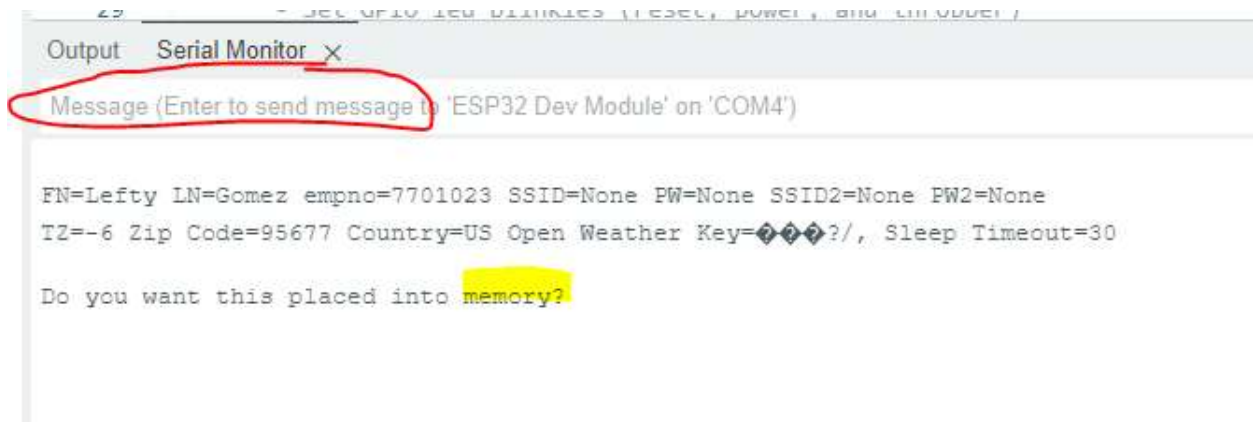
The badge will report lots of errors about Wifi. Ignore these until a message like “Update_Full...” happens. The badge tries hard to connect to Wifi. There is no WiFi configured, hence the noise.

Once the “Update_...” message happen, the device is behaving normally.

Enable the Configuration prompts from the device

While the badge is running, gently/carefully set ALL of the white lever switches DOWN (“ON”).

As soon as these switches are set, there should be a prompt on the serial monitor windows.



IMPORTANT: As soon as you see this prompt, TURN ANY ONE OF THE WHITE SWITCHES OFF. Set the switches to anything except “all on”. It is best to do this right away. Otherwise, the prompt will continue until you do reset it. It only does what it is told to do! 😊

The answer is “N” or No. This first time. You want to change these empty values. So responde “N” to the message are. See above diagram circled in red. Place the cursor there and answer “N”

From here on out, there are several prompts. For each, you must respond accurately. If you make a mistake on one, you have to do them ALL over again. You will rarely need to change this, so it should not be a problem. Several values have easy defaults.

Enter first name – Enter any characters and press enter (example here is “Keith” typed into the message are and pressed enter).

```
Output  Serial Monitor x
Keith

FN=Lefty LN=Gomez empno=7701023 SSID=None PW=None SSID2=None PW2=None
TZ=-6 Zip Code=95677 Country=US Open Weather Key=□□□?/, Sleep Timeout=30

Do you want this placed into memory?
Gomez
Gomez

FN=Lefty LN=Gomez empno=7701023 SSID=None PW=None SSID2=None PW2=None

Do you want this placed into memory?

n
Rejected. Try again.

Enter first name:
```

Enter last name – Enter any characters

```
Output  Serial Monitor x
Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')

FN=Lefty LN=Gomez empno=7701023 SSID=None PW=None SSID2=None PW2=None
TZ=-6 Zip Code=95677 Country=US Open Weather Key=□□□?/, Sleep Timeout=30

Do you want this placed into memory?
Gomez
Gomez

FN=Lefty LN=Gomez empno=7701023 SSID=None PW=None SSID2=None PW2=None

Do you want this placed into memory?

n
Rejected. Try again.

Enter first name:Keith
Enter last name:
```

The rest of the prompts I will describe. No need for images.

- Employee number (any value) – Any characters but usually an integer
- Primary Wifi SSID – SSID for connection.
- Primary Wifi Password – password for the primary SSID
- Backup WiFi SSID – A second Wifi option – Place a dummy if not used e.g. NONE
- Backup WiFi Password – Second WiFi Password or none
- TZ (GMT offset in hours) – Enter signed integer for number of hours away from GMT. Eastern US is -5, Central US is -6, etc.
- Zip code 6 digits. This is not currently used. You can default by pressing enter (no value)
- Weather country – Same. Not yet used. Press enter to default to US
- Open weather forecast key – Not used yet. Any value.
- Sleep timer – Integer 0-9999. Number of minutes before the device turns off the LEDs (sleep). Saves battery after this many minutes. Default is 60.
- Do you want to place this into memory? Check the values. Respond accordingly. If No then the prompts start again.

If the device goes back into prompt mode after you answered “Y/yes”, then you forgot to turn off the switches. Turn them off. You can then simply reset the device (power cycle or press the reset button).

RESTART THE DEVICE! You must restart the device in order for the values to be used in the display. You can turn the device off/on, or press the reset button next to the power slide switch.

Step 6

Test it out.

See operating instructions provided on the Github site.



Final Step

Enjoy!

I am at nonstoptalker@hpe.com or keithmoore@acm.org.